

Microservices using .NET Core Syllabus

OVERVIEW:

Microservices covers containerization using Docker, including Docker basics, custom images, and Docker Compose. The syllabus delves into Microservices architecture, comparing it with monolithic architecture and SOA. It includes practical demonstrations using an eStoreApplication to illustrate various concepts. Advanced topics such as securing microservices, communication between services, Domain Driven Design, CQRS, and deployment using Kubernetes are also covered.

COURSE OBJECTIVES:

- ❖ Understand containerization and Docker fundamentals
- ❖ Learn to develop and deploy .NET Core applications using Docker
- ❖ Grasp the concepts of Microservices Architecture and its benefits
- ❖ Compare Microservices with Monolithic Architecture and SOA
- ❖ Implement CRUD operations in a microservices-based application
- ❖ Integrate Swagger for API documentation and testing
- ❖ Create a user interface service for microservices
- ❖ Deploy microservices using Docker containers
- ❖ Implement security measures for microservices, including authentication and authorization
- ❖ Explore synchronous and asynchronous communication between microservices
- ❖ Apply Domain Driven Design (DDD) and Command Query Responsibility Segregation (CQRS) patterns
- ❖ Understand and implement event sourcing
- ❖ Learn strategies for handling failures in microservices
- ❖ Gain practical experience with Azure Kubernetes Service (AKS) for deploying microservices

PRE-REQUISITES:

- ❖ Solid understanding of C# and .NET Core framework
- ❖ Familiarity with ASP.NET Core MVC and Web API development
- ❖ Basic knowledge of SQL Server and database operations
- ❖ Understanding of RESTful API concepts
- ❖ Familiarity with version control systems (e.g., Git)
- ❖ Basic understanding of cloud computing concepts
- ❖ Familiarity with JSON and YAML formats

- ❖ Basic knowledge of HTML, CSS, and JavaScript
- ❖ Understanding of fundamental software design patterns
- ❖ Familiarity with containerization concepts (beneficial but not mandatory)
- ❖ Basic understanding of authentication and authorization concepts
- ❖ Familiarity with asynchronous programming in C#
- ❖ Basic knowledge of messaging systems and queues
- ❖ Understanding of fundamental web security concepts

WHO CAN DO THIS COURSE:

This course is ideal for professionals who have a strong foundation in .NET development and are looking to advance their skills in modern application architecture and deployment techniques.

- ❖ Software developers with experience in .NET technologies
- ❖ Backend developers looking to transition to microservices architecture
- ❖ Full-stack developers interested in modern application architectures
- ❖ DevOps engineers wanting to understand microservices deployment
- ❖ IT professionals with a strong programming background in C#
- ❖ System architects interested in designing scalable, distributed systems
- ❖ Cloud engineers looking to expand their knowledge of containerization and orchestration
- ❖ Software engineers aiming to modernize legacy applications
- ❖ Recent computer science graduates with a solid understanding of web development
- ❖ QA engineers interested in testing microservices-based applications
- ❖ Technical team leads or managers overseeing microservices projects
- ❖ Freelance developers looking to enhance their skills in cutting-edge technologies

TRAINING AT DECCANSOFT:

- ❖ Our esteemed trainer, Mr. Sandeep Soni, brings over 27 years of industry expertise to this course, personally crafting its content.
- ❖ Mr. Soni's teaching approach is both simple and thorough, ensuring a clear grasp of Microservices concepts from the ground up.
- ❖ Each topic is illustrated with practical demos and real-world case studies, fostering a comfortable learning experience.
- ❖ You'll seamlessly integrate all learned concepts into real-time scenarios, preparing you for practical application in your role.

MICROSERVICES COURSE OUTLINE:

Module 1: Microservices Architecture Introduction

Microservices architecture is a modern approach to software development that breaks down complex applications into smaller, independent services. Each service focuses on a specific business capability and can be developed, deployed, and scaled independently. This architectural style offers improved flexibility, scalability, and maintainability compared to traditional monolithic systems.

- ❖ Understanding Monolithic Architecture
 - What are Monolithic Applications
 - Deploying
 - Containerizing using Docker
 - Scaling Applications
 - Managing State and Data
 - Benefits and Drawbacks of Monolithic Architecture
- ❖ Microservices Architecture
 - What are Microservices
 - Monolithic vs Microservices Architecture
 - Characteristics of Microservices Architecture
 - Benefits of using Microservices Architecture
 - Microservices Design Principles.
- ❖ SOA vs. Microservice
- ❖ Handling Data in Microservices
- ❖ Logical architecture versus physical architecture
- ❖ Communication between Microservices
 - Synchronous Communication across Microservices.
 - Asynchronous communication across Microservices.
- ❖ Microservices Patterns
 - Domain Driven Design (DDD)
 - Command and Query Responsibility Separation (CQRS)
 - Event Sourcing
 - API Gateway Pattern
- ❖ Creating Composite UI with Microservices
- ❖ Role of Orchestration Servers
- ❖ Drawback of Microservices

Module 2: Basic Project Setup and Performing CRUD Operations

Setting up a microservice-based application is a crucial skill in modern software development. This guide will walk you through the process of creating a robust microservice architecture, focusing on implementing CRUD operations, utilizing domain-driven design principles, and integrating essential tools like Swagger.

- ❖ Creating a Solution and Project Layout
- ❖ Implementing a CRUD microservice
- ❖ Writing Domain Classes and Controllers
- ❖ Data Context Class and Data Seeding
- ❖ Using Repository Classes
- ❖ Swagger and Swashbuckle Integration
- ❖ Practical Demonstration using eStoreApplication
 - Product Catalog Service with SQL Server
 - Invoking both services using Swagger UI

Module 3: Building Client for using Microservices

Building a user interface (UI) service is a critical component of a microservice-based application. This section will guide you through the process of creating an ASP.NET MVC project to serve as the front end for your microservices.

- ❖ Adding ASP.NET MVC Project
- ❖ Writing Model Classes
- ❖ Writing Service Classes
- ❖ Building Web Controller and Views
- ❖ Practical Demonstration using eStoreApplication
- ❖ Writing Backend For Frontend (BFF)
 - Writing Backend For Frontend (BFF)
 - UI Microservice to display Product Catalog

Module 4: Hosting Microservices using Docker Containers

Docker containers provide an efficient and consistent way to deploy and manage microservices. This section will cover the process of containerizing your microservices, allowing for improved scalability, portability, and isolation.

- ❖ Adding Docker Support to the Microservice Application

- ❖ Creating a Docker file
- ❖ Designing and Developing Multi-Container Microservices
- ❖ Database Connection string and environment variables in Docker containers
- ❖ Handling Configuration Data
- ❖ Use a database server running as a container
- ❖ Practical Demonstration using eStoreApplication
 - Creating an SQL Server database container
 - Create a docker images for Product Catalog Microservices
 - Create a docker image for UI Microservice
 - Writing a YAML for deploying and executing the application

Module 5: Using Redis Cache in Microservices

Implementing Redis Cache in microservices can significantly enhance performance and scalability. This walkthrough will guide you through integrating Redis Cache into your microservice architecture. We'll cover project setup, model creation, and controller service implementation.

- ❖ Walkthrough – Using Redis Cache in Microservices
 - Project Setup
 - Adding Models
 - Adding Controller Service
 - Docker-compose
- ❖ Editing WebMVC (Client) Application

Module 6: Understanding Authentication and Authorization

Authentication and authorization are crucial aspects of securing microservices architectures. This section delves into the fundamentals of these security mechanisms, exploring modern protocols like OAuth2 and OpenID Connect.

- ❖ The Big Picture
- ❖ Basic Authenticate with JWT Bearer Token
- ❖ Understanding OAuth2 and OpenIdConnect
- ❖ Using IdentityServer4 Authorization Server
- ❖ Authorization Grant Types
 - Authorization Code
 - Implicit
 - Client Credentials

- Resource Owner Password Credentials (ROPC)
- ❖ Refresh Tokens
- ❖ Authentication between Microservices
- ❖ Implementing Role based and Policy based Authorization
- ❖ Practical Demonstration using eStoreApplication
 - Adding Authentication Microservice to Solution
 - Securing Product Catalog Microservice
 - Accessing secure Product Catalog in Swagger UI
 - Accessing secure Product Catalog in Client Application

Module 7: IdentityServer4

IdentityServer4 is a powerful framework for implementing authentication and authorization in microservices architectures. This section will guide you through various aspects of using IdentityServer4, starting with securing API access using client credentials. with IdentityServer4.

- ❖ IdentityServer4
- ❖ Access Secure API using Grant Type
- ❖ Adding UI with OpenID Connect Support
- ❖ API Access using Access Token
- ❖ Using ASP.NET Core Identity
- ❖ Adding Support for External Authentication

Module 8: Implementing Security in Microservices Application

Implementing security in a microservices application is crucial for protecting sensitive data and ensuring authorized access. This section will guide you through the process of securing a microservices ecosystem, focusing on creating an Identity Microservice as the central authentication server.

- ❖ Identity Microservice as Authentication Server
- ❖ Securing Product Catalog Microservices
- ❖ Securing Basket Service Microservices
- ❖ Integrating Security in Web MVC Microservice Application
- ❖ Role Based Security
- ❖ Docker Support

Module 9: API Gateway Integration

API Gateway integration is a crucial component in microservices architecture, providing a centralized entry

point for client requests. This section explores the concept of API Gateways and their benefits in managing microservices communication. We'll focus on implementing a gateway using Ocelot, a popular .NET API Gateway.

- ❖ About API Gateway
- ❖ Building Gateway Using Ocelot
- ❖ Implementing Security in Ocelot Gateway Service using IdentityServer4

Module 10: Using Integration Event in Microservice Communication

Effective communication between microservices is essential for building a robust and scalable application. This section explores both synchronous and asynchronous communication patterns in microservices architecture. We'll examine the use of REST APIs for direct, real-time interactions and Service Bus for decoupled, message-based communication

- ❖ Synchronous Communication using REST API
- ❖ Asynchronous Communication using Service Bus
- ❖ Practical Demonstration using eStoreApplication
 - Adding Shopping Cart Service with Redis Cache
 - Adding Product to Shopping Cart Service for logged-in client
 - Show Shopping Cart using View Component
 - Create a docker images for Product Catalog Microservices
 - Asynchronous Communicating with Notification Service and Service Bus Queue

Module 11: Implementing CQRS Pattern

The Command Query Responsibility Segregation (CQRS) pattern is an architectural approach that separates read and write operations in a system. This section explores the CQRS pattern and its applications in microservices architecture.

- ❖ Command Query Responsibility Segregation (CQRS)
 - When to use CQRS pattern
- ❖ Implementing reads/queries in a CQRS microservice
- ❖ Implementing Update Model in a CQRS microservice

Module 12: Implementing Domain Driven Design Pattern

Domain-Driven Design (DDD) is a powerful approach to building complex software systems, particularly suited for microservices architectures. This section explores the fundamental concepts of DDD, including domains, subdomains, and bounded contexts.

- ❖ About Domain and Domain Driven Design (DDD)
 - What is Domain?
 - What is Subdomain
 - Types of Subdomains
 - What is Domain Driven Design (DDD)
 - Three Design Pillars in DDD
- ❖ What Bounded Context
 - Bounded Context vs Subdomain
 - Teams and Source Code Repositories
- ❖ Ubiquitous Language
- ❖ Layered Architecture in DDD microservices
- ❖ Implementing the Command and Command Handler patterns
- ❖ Domain Events

Module 13: Implement Resilient Applications

Building resilient applications is crucial in microservices architecture to ensure system stability and reliability. This section focuses on implementing resilience in your microservices, with a particular emphasis on handling partial failures. We'll explore various strategies to manage these failures effectively, introducing Polly as a robust resilience framework.

- ❖ Implement Resilient Applications
 - Handling Partial Failures
 - Strategies to handle partial failure
- ❖ About Polly
- ❖ Communication with Polly and .NET HttpClientFactory
- ❖ Circuit Breakers
- ❖ Health Monitoring